VisionGuard

AI-Generated Image Classification and Deployment

Student: HASSAN
Roll number: 22L-6851
GitHub: https://github.com/VeryFASTstudent/22L-6851_A2_Ops
DagsHub: https://dagshub.com/VeryFASTstudent/22L-6851_A2_Ops
Live URL: To be added after cloud deployment

This report documents the reproducible local implementation, dataset audit, model training, evaluation artifacts, experiment tooling, Flask service, database trail, and container configuration. Cloud credentials, public deployment, and account screenshots remain operator-owned evidence.

1. Environment and dataset

The project uses Python 3.11, TensorFlow 2.18.0, Flask 3.1.0, MLflow 2.19.0, and CPU execution. The dataset was copied from the supplied archive into data/dataset and tracked with DVC through data/dataset.dvc.

class	count	width_min	width_median	width_max	height_min	height_median	height_max	size_min	size_median	size_max
AiArtData	539	225	1024.0	6000	135	800.0	4001	11738	199760.0	5626183
RealArt	435	183	940.0	8495	158	720.0	7091	11311	144982.0	10195352

2. Visual EDA

The supplied data contains 539 AI-art images and 435 readable real-art images in the generated EDA summary. Width, height, and file-size distributions are broad, so the input pipeline resizes images to 224x224 while preserving three color channels. Resizing standardizes the tensor shape but can distort aspect ratio; this is a documented limitation.

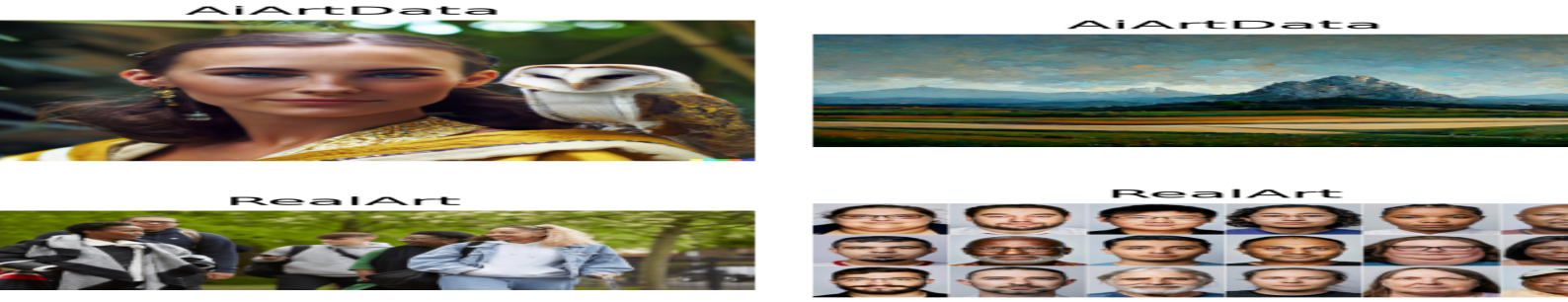


Figure 1. Representative images from both classes.

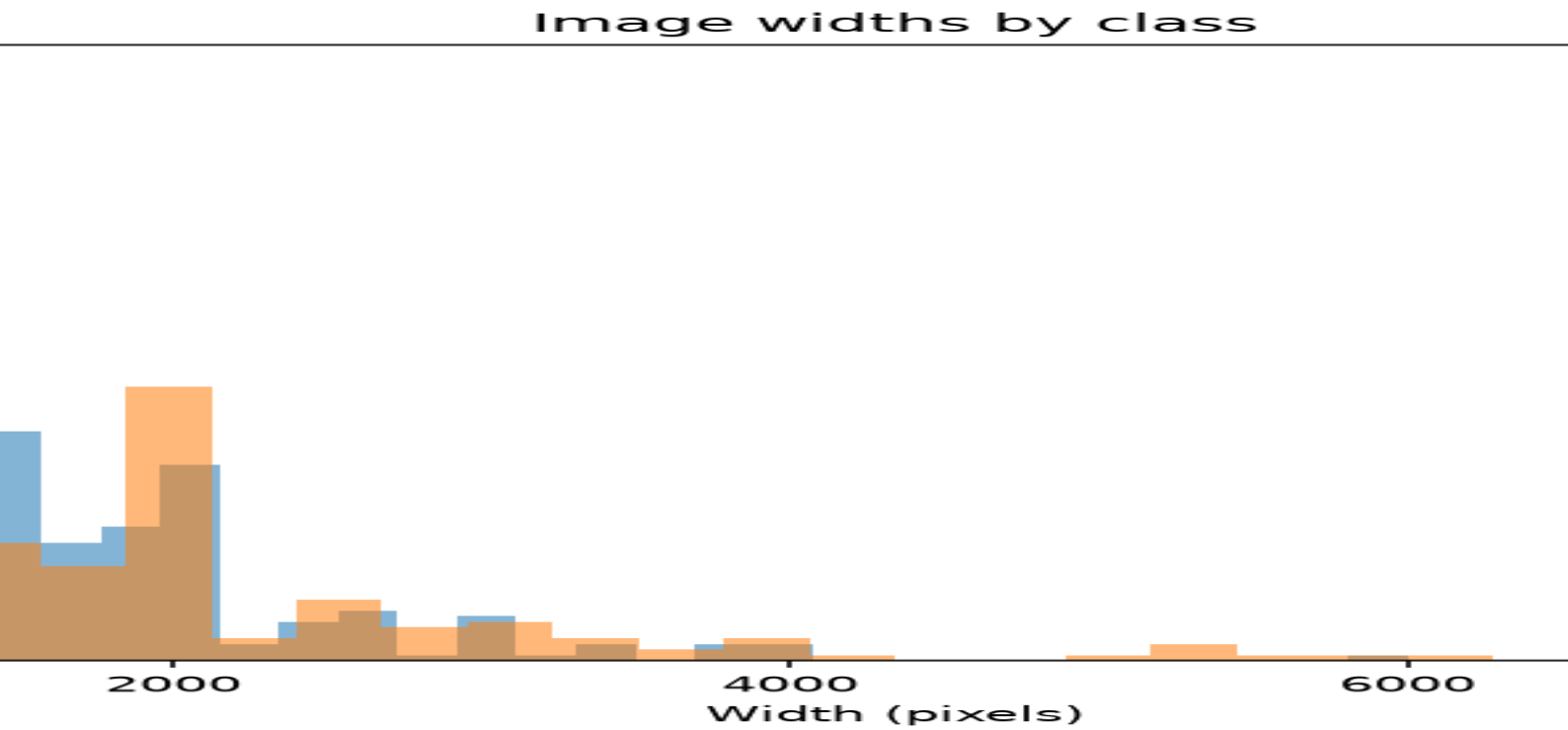


Figure 2. Width distribution by class.

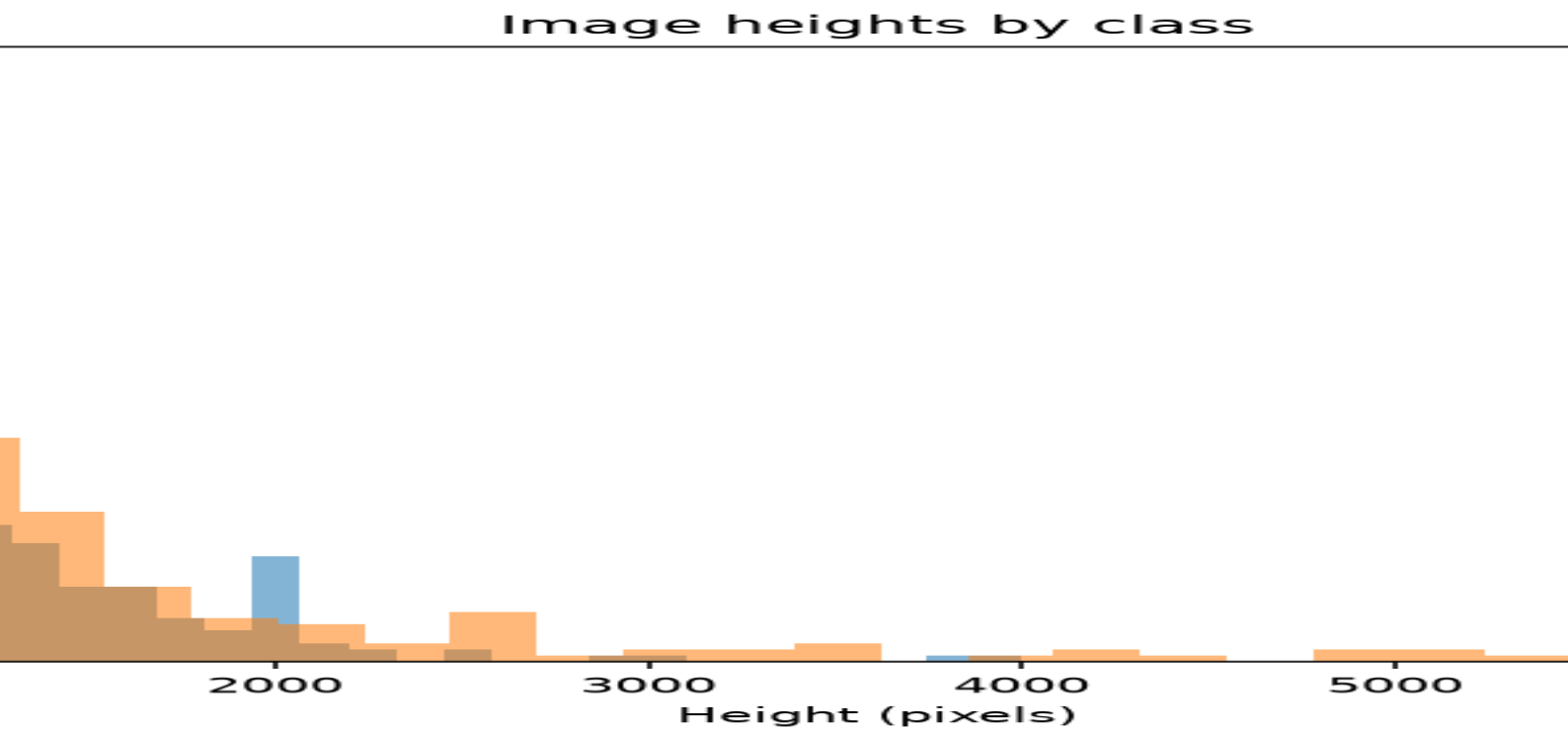


Figure 3. Height distribution by class.

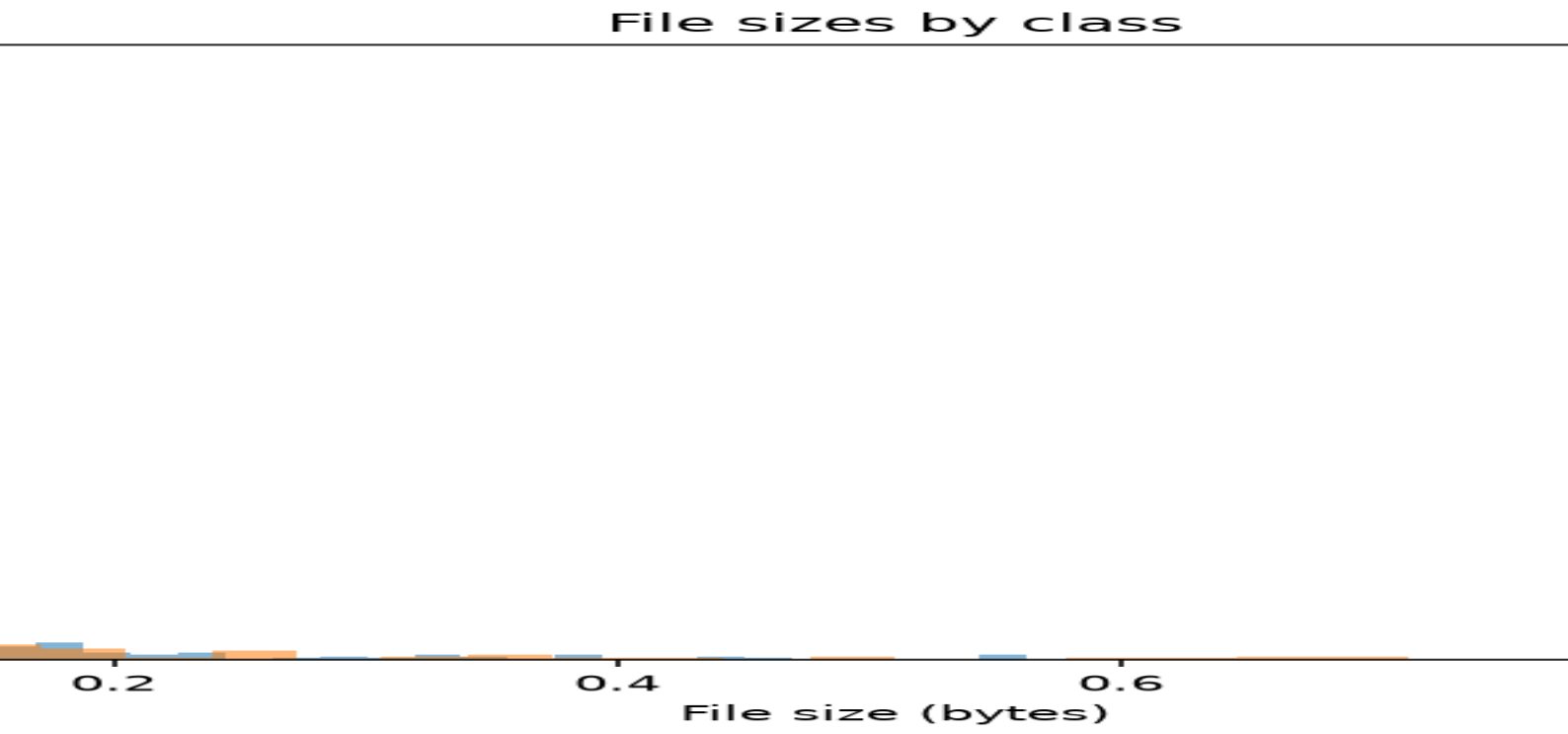
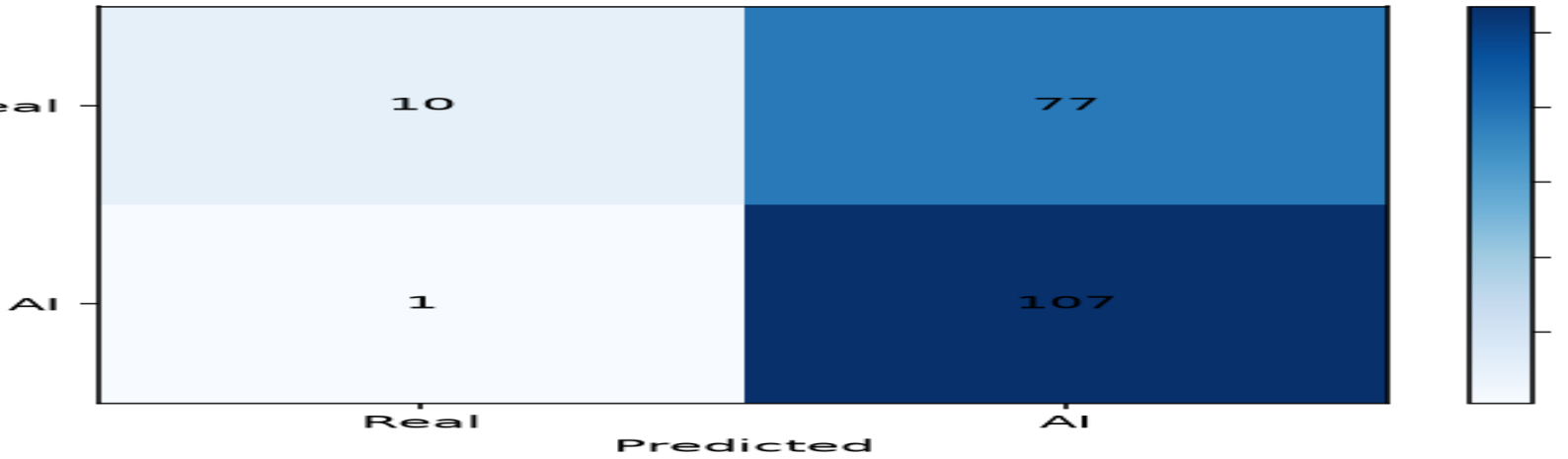


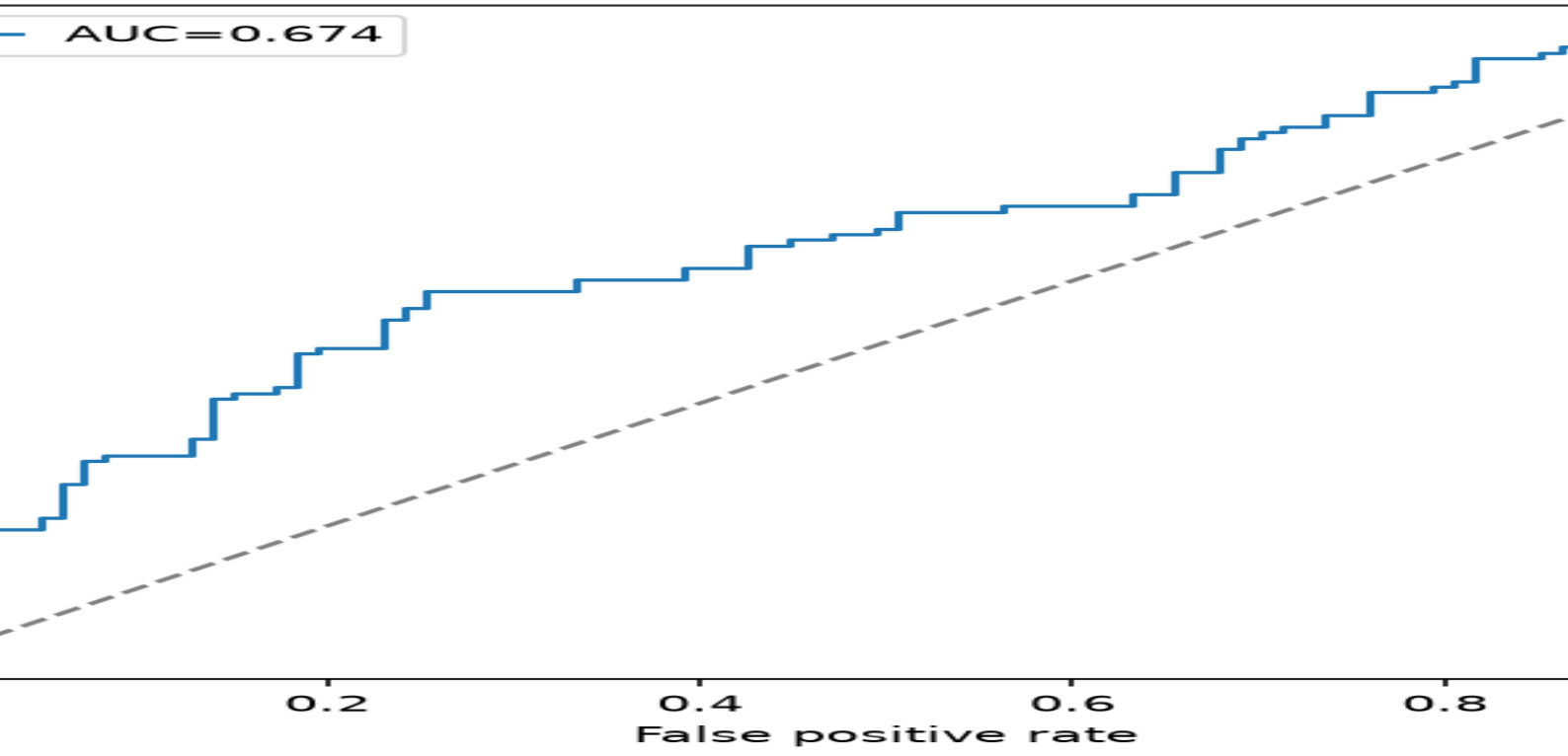
Figure 4. File-size distribution by class.

3. Models and evaluation

The baseline model is a custom CNN with convolution, batch normalization, pooling, global average pooling, dropout, and a sigmoid output. A MobileNetV2 transfer-learning model was also trained. Training uses a fixed seed, augmentation, early stopping, learning-rate reduction, and checkpointing. A later correction replaced directory-tail validation with a stratified file-level split so both classes are represented.

Metric	Champion result
threshold	0.3600
accuracy	0.6000
precision	0.5815
recall	0.9907
f1	0.7329
roc_auc	0.6741





4. Tuning and experiment tracking

The Keras Tuner toy search completed eight Fashion-MNIST trials and recorded the best units and learning rate in artifacts/tuner/best_hyperparameters.txt. The MLflow comparison script logs model paths, image size, seed, dataset pointer, evaluation metrics, and JSON artifacts. Set MLFLOW_TRACKING_URI to the DagsHub MLflow endpoint to repeat the same runs remotely.

Best toy tuner settings: {'units': 128, 'learning_rate': 0.0019526565349868704}

5. Serving and operations

The Flask service loads the champion model once at startup, validates extensions and decodability, does not persist uploaded images, records every successful prediction, and exposes /health and /history. Docker Compose provides a web service and an internal PostgreSQL service with a named volume and health-gated startup. GitHub Actions runs tests, linting, and a Docker build.

6. Submission evidence still required

The following must be captured by the student because they depend on personal accounts or external infrastructure: GitHub Actions run, DagsHub project and registry screenshots, DVC remote screenshot, cloud VM setup, public HTTPS URL, Postman request, and the final live health response. The live URL should replace the placeholder on the cover page after deployment.

Generated from the local VisionGuard workspace. Final commit: see git rev-parse HEAD